`any_view`

Document #:	P3411R0
Date:	2024-07-06
Project:	Programming Language C++
Audience:	SG9, LEWG

Reply-to:	Hui Xie <hui.xie1990@gmail.com> S. Levent Yilmaz <levent.yilmaz@gmail.com> Louis Dionne <ldionne.2@gmail.com>

Contents

§ 1 Revision History

§ 1.1 R0

— Initial revision.

§ 2 Abstract

This paper proposes a new type-erased view: `std::ranges::any_view`. That type-erased view allows customizing the traversal category of the view, its value type and a few other properties. For example:

```
class MyClass {
  std::unordered_map<Key, Widget> widgets_;
public:
  std::ranges::any_view<Widget> getWidgets();
};

std::ranges::any_view<Widget> MyClass::getWidgets() {
  return widgets_ | std::views::values
                  | std::views::filter(myFilter);
}
```

§ 3 Motivation

Since being merged into C++20, the Ranges library has gained an increasingly rich and expressive set of views. For example,

```
// in MyClass.hpp
class MyClass {
  std::unordered_map<Key, Widget> widgets_;
public:
  auto getWidgets() {
```

```
    return widgets_ | std::views::values
                    | std::views::filter([](const auto&){ /*...*/ });
  }
};
```

While such use of ranges is exceedingly convenient, it has the drawback of leaking implementation details into the interface. In this example, the return type of the function essentially bakes the implementation of the function into the interface.

In large applications, such liberal use of `std::ranges` can lead to increased header dependencies and often a significant compilation time penalty.

Attempts to separate the implementation into its own translation unit, as is a common practice for non-templated code, are futile in this situation. The return type of the above definition of `getWidgets` is:

```
std::ranges::filter_view<
  std::ranges::elements_view<
    std::ranges::ref_view<std::unordered_map<Key, Widget>>,
    1>,
  MyClass::getWidgets()::<lambda(const auto:11&)> >
```

While this type is already difficult to spell once, it is much harder and more brittle to maintain it as the implementation or the business logic evolves. These challenges for templated interfaces are hardly unique to ranges: the numerous string types in the language and lambdas are some common examples that lead to similar challenges.

Type-erasure is a very popular technique to hide the concrete type of an object behind a common interface, allowing polymorphic use of objects of any type that model a given concept. In fact, it is a technique commonly employed by the standard. `std::string_view` `std::function` and `std::function_ref`, and `std::any` are the type-erased facilities for the examples above.

`std::span<T>` is another type-erasure utility recently added to the standard; and is closely related to ranges in fact, by allowing type-erased *reference* of any underlying *contiguous* range of objects.

In this paper, we propose to extend the standard library with `std::ranges::any_view`, which provides a convenient and generalized type-erasure facility to hold any object of any type that satisfies the `ranges::view` concept itself. `std::ranges::any_view` also allows further refinement via customizable constraints on its traversal categories and other range characteristics.

§ 4 Design Space and Prior Art

Designing a type like `any_view` raises a lot of questions.

Let's take `std::function` as an example. At first, its interface seems extremely simple: it provides `operator()` and users only need to configure the return type and argument types

of the function. In reality, `std::function` makes many other decisions for the user:

— Are `std::function` and the callable it contains `copyable`?

— Does `std::function` own the callable it contains?

— Does `std::function` propagate const-ness?

After answering all these questions we ended up with several types:

— `function`

— `move_only_function`

— `function_ref`

— `copyable_function`

The design space of `any_view` is a lot more complex than that:

— Is it an `input_range`, `forward_range`, `bidirectional_range`, `random_access_range`, or a `contiguous_range` ?

— Is the range `copyable` ?

— Is it a `sized_range` ?

— Is it a `borrowed_range` ?

— Is it a `common_range` ?

— What is the `range_reference_t` ?

— What is the `range_value_t` ?

— What is the `range_rvalue_reference_t` ?

— What is the `range_difference_t` ?

— Is the `range` const-iterable?

— Is the iterator `copyable` for `input_iterator`?

— Is the iterator equality comparable for `input_iterator`?

— Do the iterator and sentinel types satisfy `sized_sentinel_for<S, I>`?

We can easily get a combinatorial explosion of types if we follow the same approach we did for `std::function`. Fortunately, there is prior art to help us guide the design.

§ 4.1 Boost.Range `boost::ranges::any_range`

The type declaration is:

```
template<
    class Value
  , class Traversal
  , class Reference
```

```
  , class Difference
  , class Buffer = any_iterator_default_buffer
>
class any_range;
```

Users will need to provide `range_reference_t`, `range_value_t` and `range_difference_t`. `Traversal` is equivalent to `iterator_concept`, which decides the traversal category of the range. Users don't need to specify `copyable`, `borrowed_range` and `common_range`, because all Boost.Range ranges are `copyable`, `borrowed_range` and `common_range`. `sized_range` and `range_rvalue_reference_t` are not considered in the design.

§ 4.2 range-v3 `ranges::views::any_view`

The type declaration is:

```
enum class category
{
    none = 0,
    input = 1,
    forward = 3,
    bidirectional = 7,
    random_access = 15,
    mask = random_access,
    sized = 16,
};

template<typename Ref, category Cat = category::input>
struct any_view;
```

Here `Cat` handles both the traversal category and `sized_range`. `Ref` is the `range_reference_t`. It does not allow users to configure the `range_value_t`, `range_difference_t`, `borrowed_range` and `common_range`. `copyable` is mandatory in range-v3.

§ 5 Proposed Design

This paper proposes the following interface:

```
enum class any_view_options
{
    input = 1,
    forward = 3,
    bidirectional = 7,
    random_access = 15,
    contiguous = 31,
    sized = 32,
    borrowed = 64,
    move_only = 128
};
```

```
template <class Value,
          any_view_options Opts = any_view_options::input,
          class Ref = Value &,
                                                  class     RValueRef     =
        add_rvalue_reference_t<remove_reference_t<Ref>>,
          class Diff = ptrdiff_t>
class any_view {
  class iterator; // exposition-only
  class sentinel; // exposition-only

  template <class View>
    requires(!std::same_as<View, any_view> && std::ranges::view<View> &&
             view_options_constraint<View>())
  any_view(View view);

          any_view(const     any_view     &)     requires     (!(Opts     &
        any_view_options::move_only));

  any_view(any_view &&) = default;

      any_view   &operator=(const   any_view   &)   requires   (!(Opts   &
        any_view_options::move_only));

  any_view &operator=(any_view &&);

  iterator begin();
  sentinel end();

  size_t size() const requires(Opts & any_view_options::sized);
};

template <class Value, any_view_options Opts, class Ref, class RValueRef,
          class Diff>
inline constexpr bool
    enable_borrowed_range<any_view<Value, Opts, Ref, RValueRef, Diff>> =
        Opts & any_view_options::borrowed;
```

The intent is that users can select various desired properties of the `any_view` by `bitwise-or`ing them. For example:

```
using           MyView           =           std::ranges::any_view<Widget,
        std::ranges::any_view_options::bidirectional                       |
        std::ranges::any_view_options::sized>;
```

§ 6 Other Design Considerations

§ 6.1 Should the first argument be `Ref` or `Value`?

If the first template parameter is `Ref`, we have:

```
template <class Ref,
          any_view_options Opts = any_view_options::input,
          class Value = decay_t<Ref>>
```

For a range with a reference to `int`, one would write

```
any_view<int&>
```

And for a `const` reference to `int`, one would write

```
any_view<const int&>
```

In case of a generator range, e.g a `transform_view` which generates pr-value `int`, the usage would be

```
any_view<int>
```

However, it is possible that the user uses `any_view<string>` without realizing that they specified the reference type and they now make a copy of the `string` every time when the iterator is dereferenced.

Instead, if the first template parameter is `Value`, we have:

```
template <class Value,
          any_view_options Opts = any_view_options::input,
          class Ref = Value&>
```

For a range with a reference to `int`, it would be less verbose

```
any_view<int>
```

However, in order to have a `const` reference to `int`, one would have to explicitly specify the `Value`, the any_view_options and finally the `Ref`, i.e.

```
any_view<int, any_view_options::input, const int&>
```

This is a bit verbose. In the case of a generator range, one would need to do the same:

```
any_view<int, any_view_options::input, int>
```

Author Recommendation: Even though the first option is less verbose in few cases, it might create unnecessary copies without user realizing it. The author recommends the second option.

§ 6.2 Name of the `any_view_options`

`range-v3` uses the name `category` for the category enumeration type. However, the authors believe that the name `std::ranges::category` is too general and it should be reserved for more general purpose utility in ranges library. Therefore, the authors recommend a more specific name: `any_view_options`.

§ 6.3 `constexpr` Support

We do not require `constexpr` in order to allow efficient implementations using e.g. SBO. There is no way, with the current working draft, to construct an object of different type on

a `unsigned char[N]` or `std::byte[N]` buffer in `constexpr` context.

§ 6.4 Move-only `view` Support

Move-only `view` is worth supporting as we generally support them in `ranges`. We propose to have a configuration template parameter `any_view_options::move_only` to make the `any_view` conditionally move-only. This removes the need to have another type `move_only_any_view` as we did for `move_only_function`.

We also propose that by default, `any_view` is copyable and to make it move-only, the user needs to explicitly provide this template parameter `any_view_options::move_only`.

§ 6.5 Move-only `iterator` Support

In this proposal, `any_view::iterator` is an exposition-only type. It is not worth making this `iterator` configurable. If the `iterator` is only `input_iterator`, we can also make it a move-only iterator. There is no need to make it copyable. Existing algorithms that take "input only" iterators already know that they cannot copy them.

§ 6.6 Is `any_view_options::contiguous` Needed ?

`contiguous_range` is still useful to support even though we have already `std::span`. But `span` is non-owning and `any_view` owns the underlying `view`.

§ 6.7 Is `any_view` const-iterable?

We cannot make `any_view` unconditionally const-iterable. If we did, `views` with cache-on-begin, like `filter_view` and `drop_while_view` could no longer be put into an `any_view`.

One option would be to make `any_view` conditionally const-iterable, via a configuration template parameter. However, this would make the whole interface much more complicated, as each configuration template parameter would need to be duplicated. Indeed, associated types like `Ref` and `RValueRef` can be different between `const` and non-`const` iterators.

For simplicity, the authors propose to make `any_view` unconditionally non-const-iterable.

§ 6.8 `common_range` support

Unconditionally making `any_view` a `common_range` is not an option. This would exclude most of the Standard Library `views`. Adding a configuration template parameter to make `any_view` conditionally `common_range` is overkill. After all, if users need `common_range`, they can use `my_any_view | views::common`. Furthermore, supporting this turns out to add substantial complexity in the implementation. The authors believe that adding `common_range` support is not worth the added complexity.

§ 6.9 `borrowed_range` support

Having support for `borrowed_range` is simple enough:

— 1. Add a template configuration parameter

— 2. Specialise the `enable_borrowed_range` if the template parameter is set to `true`

Therefore, we recommend conditional support for `borrowed_range`. However, since `borrowed_range` is not a very useful concept in general, this design point is open for discussion.

§ 6.10 Valueless state of `any_view`

We propose providing the strong exception safety guarantee in the following operations: swap, copy-assignment, move-assignment and move-construction. This means that if the operation fails, the two `any_view` objects will be in their original states. If the underlying view's move constructor (or move-assignment operator) is not `noexcept`, the only way to achieve the strong exception safety guarantee is to avoid calling these operations altogether, which requires `any_view` to hold its underlying object on the heap so it can implement these operations by swapping pointers. This means that any implementation of `any_view` will have an empty state, and a "moved-from" `any_view` will be in that state.

§ 6.11 ABI Stability

As a type intended to exist at ABI boundaries, ensuring the ABI stability of `any_view` is extremely important. However, since almost any change to the API of `any_view` will require a modification to the vtable, this makes `any_view` somewhat fragile to incremental evolution. This drawback is shared by all C++ types that live at ABI boundaries, and while we don't think this impacts the livelihood of `any_view`, this evolutionary challenge should be kept in mind by WG21.

§ 6.12 Performance

One of the major concerns of using type erasure is the performance cost of indirect function calls and their impact on the ability for the compiler to perform optimizations. With `any_view`, every iteration will have three indirect function calls:

```
++it;
it != last;
*it;
```

While it may at first seem prohibitive, it is useful to remember the context in which `any_view` is used and what the alternatives to it are.

§ 6.12.1 A naive micro benchmark: iteration over `vector` vs `any_view`

Naively, one would be tempted to benchmark the cost of iterating over a `std::vector` and to compare it with the cost of iterating over `any_view`. For example, the following code:

```
      std::vector    v    =    std::views::iota(0,    state.range(0))    |
       std::ranges::to<std::vector>();
  for (auto _ : state) {
    for (auto i : v) {
      benchmark::DoNotOptimize(i);
    }
  }
```

vs

```
      std::vector    v    =    std::views::iota(0,    state.range(0))    |
       std::ranges::to<std::vector>();
  std::ranges::any_view<int&> av(std::views::all(v));
  for (auto _ : state) {
    for (auto i : av) {
      benchmark::DoNotOptimize(i);
    }
  }
```

-O0

```
Benchmark                                                  Time      Time vector
        Time any_view
-------------------------------------------------------------------------------
        ---------------------
[BM_vector vs. BM_AnyView]/1024                       +3.4488            10423
        46371
[BM_vector vs. BM_AnyView]/2048                       +3.3358            21318
        92432
[BM_vector vs. BM_AnyView]/4096                       +3.4224            41864
        185137
[BM_vector vs. BM_AnyView]/8192                       +3.4665            83019
        370802
[BM_vector vs. BM_AnyView]/16384                      +3.4586           166596
        742785
[BM_vector vs. BM_AnyView]/32768                      +3.4413           333311
        1480349
[BM_vector vs. BM_AnyView]/65536                      +3.4166           667125
        2946432
[BM_vector vs. BM_AnyView]/131072                     +3.4295          1335405
        5915230
[BM_vector vs. BM_AnyView]/262144                     +3.4320          2665004
        11811264
OVERALL_GEOMEAN                                       +3.4278                0
        0
```

-O2

```
Benchmark                                                  Time      Time vector
        Time any_view
-------------------------------------------------------------------------------
        -----------------
[BM_vector vs. BM_AnyView]/1024                      +14.8383              315
        4991
```

```
[BM_vector vs. BM_AnyView]/2048                  +14.9416             632
        10075
[BM_vector vs. BM_AnyView]/4096                  +15.1943            1231
        19942
[BM_vector vs. BM_AnyView]/8192                  +15.1609            2465
        39835
[BM_vector vs. BM_AnyView]/16384                 +13.8958            5386
        80235
[BM_vector vs. BM_AnyView]/32768                 +13.8638           10720
        159341
[BM_vector vs. BM_AnyView]/65536                 +13.6891           21772
        319807
[BM_vector vs. BM_AnyView]/131072                +13.5340           44363
        644768
[BM_vector vs. BM_AnyView]/262144                +13.5374           87600
        1273476
OVERALL_GEOMEAN                                  +16.0765               0
        0
```

We can see that `any_view` is 3 to 16 times slower on iteration than `std::vector`. However, this benchmark is not a realistic use case. No one would create a `vector`, immediately create a type erased wrapper `any_view` that wraps it and then iterate through it. Similarly, no one would create a lambda, immediately create a `std::function` and then call it.

§ 6.12.2 A slightly more realistic benchmark: A view pipeline vs `any_view`

Since `any_view` is intended to be used at an ABI boundary, a more realistic benchmark would separate the creation of the view in a different TU. Furthermore, most uses of `any_view` are expected to be with non-trivial view pipelines, not with e.g. a raw `std::vector`. As the pipeline increases in complexity, the relative cost of using `any_view` becomes smaller and smaller.

Let's consider the following example, where we create a moderately complex pipeline and pass it across an ABI boundary either with a `any_view` or with the pipeline's actual type:

```
// header file
struct Widget {
  std::string name;
  int size;
};

struct UI {
  std::vector<Widget> widgets_;
  std::ranges::transform_view<complicated...> getWidgetNames();
};

// cpp file
std::ranges::transform_view<complicated...> UI::getWidgetNames() {
  return widgets_ | std::views::filter([](const Widget& widget) {
           return widget.size > 10;
         }) |
         std::views::transform(&Widget::name);
}
```

And this is what we are going to measure:

```
  lib::UI ui = {...};
  for (auto _ : state) {
    for (auto& name : ui.getWidgetNames()) {
      benchmark::DoNotOptimize(name);
    }
  }
```

In the `any_view` case, we simply replace `std::ranges::transform_view<complicated...>` by `std::ranges::any_view` and we measure the same thing.

-O0

```
Benchmark                                                                  Time
        Time complicated    Time any_view
-------------------------------------------------------------------------------
        ---------------------------------
[BM_RawPipeline vs. BM_AnyViewPipeline]/1024                           +0.4290
        78469           112130
[BM_RawPipeline vs. BM_AnyViewPipeline]/2048                           +0.4051
        159225          223729
[BM_RawPipeline vs. BM_AnyViewPipeline]/4096                           +0.3568
        331276          449466
[BM_RawPipeline vs. BM_AnyViewPipeline]/8192                           +0.4022
        639566          896817
[BM_RawPipeline vs. BM_AnyViewPipeline]/16384                          +0.4148
        1267196         1792804
[BM_RawPipeline vs. BM_AnyViewPipeline]/32768                          +0.4293
        2522849         3606022
[BM_RawPipeline vs. BM_AnyViewPipeline]/65536                          +0.4199
        5078713         7211428
[BM_RawPipeline vs. BM_AnyViewPipeline]/131072                         +0.4170
        10142694        14372118
[BM_RawPipeline vs. BM_AnyViewPipeline]/262144                         +0.4358
        20386564        29270816
OVERALL_GEOMEAN                                                        +0.4120
        0               0
```

-O2

```
Benchmark                                                                  Time
        Time complicated    Time any_view
-------------------------------------------------------------------------------
        ---------------------------------
[BM_RawPipeline vs. BM_AnyViewPipeline]/1024                           +0.8066
        3504            6330
[BM_RawPipeline vs. BM_AnyViewPipeline]/2048                           +0.7136
        7339            12576
[BM_RawPipeline vs. BM_AnyViewPipeline]/4096                           +0.6746
        14841           24853
[BM_RawPipeline vs. BM_AnyViewPipeline]/8192                           +0.6424
        30177           49563
[BM_RawPipeline vs. BM_AnyViewPipeline]/16384                          +0.6538
        60751           100468
```

```
[BM_RawPipeline vs. BM_AnyViewPipeline]/32768                    +0.6524
        121345           200514
[BM_RawPipeline vs. BM_AnyViewPipeline]/65536                    +0.6582
        240378           398604
[BM_RawPipeline vs. BM_AnyViewPipeline]/131072                   +0.6861
        484220           816458
[BM_RawPipeline vs. BM_AnyViewPipeline]/262144                   +0.6234
        991733          1609940
OVERALL_GEOMEAN                                                  +0.6782
        0                0
```

This benchmark shows that `any_view` is about 40% - 70% slower on iteration, which is much better than the previous naive benchmark. However, note that this benchmark is still not very realistic. First, the type of the view pipeline is in fact so difficult to write that nobody would do it. Second, writing down the type of the view pipeline causes an implementation detail (how the pipeline is implemented) to leak into the API and the ABI of this class, which is undesirable.

As a result, most people would instead copy the results of the pipeline into a container and return that from `getWidgetNames()`, for example as a `std::vector<std::string>`. This leads us to our last benchmark, which we believe is much more representative of what people would do in the wild.

§ 6.12.3 A realistic benchmark: A copy of `vector<string>` vs `any_view`

As mentioned above, various concerns that are not related to performance like readability or API/ABI stability mean that the previous benchmarks are not really representative of what happens in the real world. Instead, people in the wild tend to use owning containers like `std::vector` as a type erasure tool for lack of a better tool. Such code would look like this:

```
// header file
struct UI {
  std::vector<Widget> widgets_;
  std::vector<std::string> getWidgetNames() const;
};

// cpp file
std::vector<std::string> UI::getWidgetNames() const {
  return widgets_ | std::views::filter([](const Widget& widget) {
           return widget.size > 10;
         }) |
                                  std::views::transform(&Widget::name)    |
         std::ranges::to<std::vector>();
}
```

-O0

```
Benchmark                                                          Time
        Time vector<string>    Time any_view
-------------------------------------------------------------------------
        -----------------------------------
```

```
[BM_VectorCopy vs. BM_AnyViewPipeline]/1024                    -0.5376
        238558           110316
[BM_VectorCopy vs. BM_AnyViewPipeline]/2048                    -0.5110
        454350           222187
[BM_VectorCopy vs. BM_AnyViewPipeline]/4096                    -0.4868
        886121           454774
[BM_VectorCopy vs. BM_AnyViewPipeline]/8192                    -0.4766
        1729318           905041
[BM_VectorCopy vs. BM_AnyViewPipeline]/16384                   -0.4834
        3462454          1788737
[BM_VectorCopy vs. BM_AnyViewPipeline]/32768                   -0.4858
        7006102          3602475
[BM_VectorCopy vs. BM_AnyViewPipeline]/65536                   -0.4777
        13741174          7176723
[BM_VectorCopy vs. BM_AnyViewPipeline]/131072                  -0.4792
        27501856         14321826
[BM_VectorCopy vs. BM_AnyViewPipeline]/262144                  -0.4838
        55950048         28883803
OVERALL_GEOMEAN                                                -0.4917
        0                0
```

-O2

```
Benchmark                                                              Time
        Time vector<string>    Time any_view
-------------------------------------------------------------------------
        ---------------------------------
[BM_VectorCopy vs. BM_AnyViewPipeline]/1024                    -0.8228
        35350             6264
[BM_VectorCopy vs. BM_AnyViewPipeline]/2048                    -0.8250
        71983            12596
[BM_VectorCopy vs. BM_AnyViewPipeline]/4096                    -0.8320
        148942            25018
[BM_VectorCopy vs. BM_AnyViewPipeline]/8192                    -0.8276
        291307            50234
[BM_VectorCopy vs. BM_AnyViewPipeline]/16384                   -0.8304
        590026           100058
[BM_VectorCopy vs. BM_AnyViewPipeline]/32768                   -0.8301
        1175121           199685
[BM_VectorCopy vs. BM_AnyViewPipeline]/65536                   -0.8297
        2363963           402634
[BM_VectorCopy vs. BM_AnyViewPipeline]/131072                  -0.8340
        4841300           803467
[BM_VectorCopy vs. BM_AnyViewPipeline]/262144                  -0.8463
        10412999          1600341
OVERALL_GEOMEAN                                                -0.8310
        0                0
```

With this more realistic use case, we can see that `any_view` is 50% - 80% faster. In our benchmark, 10% of the `Widgets` were filtered out by the filter pipeline and the `name` string's length is randomly 0-30. So some of `strings` are in the SBO and some are allocated on the heap. We maintain that this code pattern is very common in the wild: making the code simple and clean at the cost of copying data, even though most of the callers don't actually need a copy of the data at all.

In conclusion, we believe that while it's easy to craft benchmarks that make `any_view` look bad performance-wise, in reality this type can often lead to better performance by

sidestepping the need to own the data being iterated over.

Furthermore, by putting this type in the Standard library, we would make it possible for implementers to use optimziations like selective devirtualization of some common operations like `for_each` to achieve large performance gains in specific cases.

§ 7 Implementation Experience

`any_view` has been implemented in [range-v3], with equivalent semantics as proposed here. The authors also implemented a version that directly follows the proposed wording below without any issue [ours].

§ 8 Wording

§ 8.1 Feature Test Macro

```

# § 9 References


[ours] Hui Xie, S. Levent Yilmaz, and Dionne Louis. A proof-of-concept implementation of `any_view`.
https://github.com/huixie90/cpp_papers/tree/main/impl/any_view

[range-v3] Eric Niebler. range-v3 library.
https://github.com/ericniebler/range-v3
```